# MODULE 6.2 PROGRAMMING ASSIGNMENT

**GitHub Repository Link:** https://github.com/devcnx/csd440

**BRITTANEYMYINTEGER.PHP**

```php
<?php
/**
 * Module 6.2 Programming Assignment
 * CSD440 Server-Side Scripting
 *
 * This program defines a BrittaneyMyInteger class that encapsulates a
 * single integer value. The class exposes:
 *   - A constructor that accepts the initial integer
 *   - Getter and setter methods for the encapsulated value
 *   - isEven(int $n) and isOdd(int $n) — accept any integer and return a
 *     boolean, as specified in the assignment prompt
 *   - isPrime() — evaluates the currently stored value
 *
 * Two instances are created and every method is exercised to verify
 * correctness. Results are rendered in an HTML table for readability.
 *
 * @author Brittaney Perry-Morgan
 * @date   2026-04-19
 */

/**
 * Encapsulates a single integer value and provides parity and primality
 * checks. Demonstrates standard PHP OOP patterns: private properties,
 * constructor assignment, accessor methods, and instance/parameter-based
 * boolean checks.
 */
class BrittaneyMyInteger
{
    /**
     * The encapsulated integer value. Declared private so external code
     * must interact with it through the getter and setter, which is the
     * whole point of encapsulation.
     *
     * @var int
     */
    private int $value;

    /**
     * Constructs a new BrittaneyMyInteger with the supplied integer.
     *
     * @param int $value The initial integer to store
     */
    public function __construct(int $value)
    {
        $this->value = $value;
    }
```

```
/**
 * Returns the currently stored integer value.
 *
 * @return int The encapsulated value
 */
public function getValue(): int
{
    return $this->value;
}

/**
 * Replaces the currently stored integer with a new value.
 *
 * @param int $value The new integer to store
 * @return void
 */
public function setValue(int $value): void
{
    $this->value = $value;
}

/**
 * Returns true when the supplied integer is even.
 *
 * Uses the modulo operator against 2. Zero is considered even, and
 * negative numbers follow the same rule (e.g., -4 is even).
 *
 * @param int $n The integer to test
 * @return bool True if $n is evenly divisible by 2
 */
public function isEven(int $n): bool
{
    return $n % 2 === 0;
}

/**
 * Returns true when the supplied integer is odd.
 *
 * Implemented as the logical inverse of isEven() so the two methods
 * cannot drift out of sync if the parity rule is ever changed.
 *
 * @param int $n The integer to test
 * @return bool True if $n is not evenly divisible by 2
 */
public function isOdd(int $n): bool
{
    return !$this->isEven($n);
}

/**
 * Returns true when the currently stored value is a prime number.
 *
 * A prime number is a whole number greater than 1 whose only positive
```

```
    * divisors are 1 and itself. The loop checks divisibility up to the
    * square root of the value — no divisor larger than sqrt(n) can
    * produce a factor that hasn't already been found.
    *
    * Edge cases:
    *   - Values less than 2 (including zero and negatives) are not prime
    *   - 2 is the only even prime and is handled as a special case so the
    *     main loop can skip all even numbers
    *
    * @return bool True if the stored value is prime, false otherwise
    */
   public function isPrime(): bool
   {
      $n = $this->value;

      if ($n < 2) {
         return false;
      }
      if ($n === 2) {
         return true;
      }
      if ($n % 2 === 0) {
         return false;
      }

      $limit = (int) floor(sqrt($n));
      for ($i = 3; $i <= $limit; $i += 2) {
         if ($n % $i === 0) {
            return false;
         }
      }
      return true;
   }
}

/**
 * Formats a boolean test result as a colored HTML span so the browser
 * output makes pass/fail obvious at a glance.
 *
 * @param bool $result The boolean to format
 * @return string      HTML markup representing the result
 */
function formatBool(bool $result): string
{
   $class = $result ? 'flag-true' : 'flag-false';
   $label = $result ? 'TRUE' : 'FALSE';
   return "<span class=\"$class\">$label</span>";
}

// Create two instances — one prime, one composite — to exercise the class
$intA = new BrittaneyMyInteger(17);
$intB = new BrittaneyMyInteger(42);
```

```
// Collected test results for display in the page body
$tests = [
    [
        'label' => 'Instance A — getValue()',
        'method' => 'getValue()',
        'result' => (string) $intA->getValue(),
    ],
    [
        'label' => 'Instance A — isEven(10)',
        'method' => 'isEven(10)',
        'result' => formatBool($intA->isEven(10)),
    ],
    [
        'label' => 'Instance A — isOdd(10)',
        'method' => 'isOdd(10)',
        'result' => formatBool($intA->isOdd(10)),
    ],
    [
        'label' => 'Instance A — isEven(7)',
        'method' => 'isEven(7)',
        'result' => formatBool($intA->isEven(7)),
    ],
    [
        'label' => 'Instance A — isOdd(7)',
        'method' => 'isOdd(7)',
        'result' => formatBool($intA->isOdd(7)),
    ],
    [
        'label' => 'Instance A — isPrime() on stored value 17',
        'method' => 'isPrime()',
        'result' => formatBool($intA->isPrime()),
    ],
    [
        'label' => 'Instance B — getValue()',
        'method' => 'getValue()',
        'result' => (string) $intB->getValue(),
    ],
    [
        'label' => 'Instance B — isEven(0)',
        'method' => 'isEven(0)',
        'result' => formatBool($intB->isEven(0)),
    ],
    [
        'label' => 'Instance B — isOdd(-3)',
        'method' => 'isOdd(-3)',
        'result' => formatBool($intB->isOdd(-3)),
    ],
    [
        'label' => 'Instance B — isPrime() on stored value 42',
        'method' => 'isPrime()',
        'result' => formatBool($intB->isPrime()),
    ],
];
```

```
// Exercise the setter on Instance B and re-run isPrime()
$intB->setValue(29);
$tests[] = [
   'label' => 'Instance B — setValue(29), then getValue()',
   'method' => 'setValue(29) → getValue()',
   'result' => (string) $intB->getValue(),
];
$tests[] = [
   'label' => 'Instance B — isPrime() on new stored value 29',
   'method' => 'isPrime()',
   'result' => formatBool($intB->isPrime()),
];

// Build a prime sweep from 1 through 20 to demonstrate isPrime() over a range
$primeSweep = [];
for ($candidate = 1; $candidate <= 20; $candidate++) {
   $probe = new BrittaneyMyInteger($candidate);
   $primeSweep[] = [
      'n' => $candidate,
      'isEven' => $probe->isEven($candidate),
      'isOdd' => $probe->isOdd($candidate),
      'isPrime' => $probe->isPrime(),
   ];
}

// Page metadata
$studentName = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 6.2 Programming Assignment';
$courseName = 'CSD440 Server-Side Scripting';
date_default_timezone_set('America/Chicago');
$date = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

   <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>MyInteger Class — <?php echo $assignmentTitle; ?></title>
      <style>
         body {
            font-family: Arial, sans-serif;
            max-width: 960px;
            margin: 40px auto;
            padding: 20px;
            background-color: #f5f5f5;
            color: #222;
         }

         h1 {
            text-align: center;
            color: #2c3e50;
```

```
}

h2 {
    color: #2c3e50;
    border-bottom: 2px solid #4a90d9;
    padding-bottom: 6px;
    margin-top: 36px;
}

.header-info {
    text-align: center;
    margin-bottom: 30px;
    color: #555;
}

.query-note {
    background: #eef4fb;
    border-left: 4px solid #4a90d9;
    padding: 10px 14px;
    margin: 12px 0;
    font-size: 0.95em;
}

table {
    width: 100%;
    border-collapse: collapse;
    background-color: #fff;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.08);
    margin-top: 8px;
}

th {
    background-color: #2c3e50;
    color: #fff;
    padding: 10px 12px;
    text-align: left;
}

td {
    padding: 10px 12px;
    border-bottom: 1px solid #ddd;
}

tr:hover {
    background-color: #f9f9f9;
}

code {
    background: #f0f0f0;
    padding: 2px 6px;
    border-radius: 3px;
    font-family: Menlo, Consolas, monospace;
    font-size: 0.9em;
```

```
        }

        .flag-true {
            color: #27ae60;
            font-weight: bold;
        }

        .flag-false {
            color: #c0392b;
            font-weight: bold;
        }

        .prime-row {
            background-color: #e8f7ee;
        }
    </style>
</head>

<body>
    <h1>MyInteger Class Test Harness</h1>
    <div class="header-info">
        <p><strong><?php echo $studentName; ?></strong></p>
        <p><?php echo $assignmentTitle; ?> — <?php echo $courseName; ?></p>
        <p><?php echo $date; ?></p>
    </div>

    <h2>Instances Under Test</h2>
    <div class="query-note">
        <p>Two instances of <code>BrittaneyMyInteger</code> are created to exercise every method:</p>
        <ul>
            <li><code>$intA = new BrittaneyMyInteger(17);</code> — a prime value</li>
            <li><code>$intB = new BrittaneyMyInteger(42);</code> — a composite value, later reassigned via
                <code>setValue(29)</code>
            </li>
        </ul>
    </div>

    <h2>Method Invocations</h2>
    <table>
        <thead>
            <tr>
                <th>Test</th>
                <th>Method Call</th>
                <th>Result</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($tests as $t): ?>
                <tr>
                    <td><?php echo htmlspecialchars($t['label']); ?></td>
                    <td><code><?php echo htmlspecialchars($t['method']); ?></code></td>
                    <td><?php echo $t['result']; ?></td>
                </tr>
```

```
            <?php endforeach; ?>
        </tbody>
    </table>

    <h2>isPrime() Sweep: 1 – 20</h2>
    <div class="query-note">
        Each candidate is wrapped in its own instance of <code>BrittaneyMyInteger</code>
        and probed with all three boolean methods. Prime rows are highlighted so the
        classic prime sequence (2, 3, 5, 7, 11, 13, 17, 19) is easy to verify at a glance.
    </div>
    <table>
        <thead>
            <tr>
                <th>n</th>
                <th>isEven(n)</th>
                <th>isOdd(n)</th>
                <th>isPrime()</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($primeSweep as $row): ?>
                <tr class="<?php echo $row['isPrime'] ? 'prime-row' : ''; ?>">
                    <td><?php echo $row['n']; ?></td>
                    <td><?php echo formatBool($row['isEven']); ?></td>
                    <td><?php echo formatBool($row['isOdd']); ?></td>
                    <td><?php echo formatBool($row['isPrime']); ?></td>
                </tr>
            <?php endforeach; ?>
        </tbody>
    </table>
  </body>

</html>
```

## DELIVERABLES

# MyInteger Class Test Harness

**Brittaney Perry-Morgan**

Module 6.2 Programming Assignment — CSD440 Server-Side Scripting

April 19, 2026

## Instances Under Test

Two instances of `BrittaneyMyInteger` are created to exercise every method:

- `$intA = new BrittaneyMyInteger(17);` — a prime value
- `$intB = new BrittaneyMyInteger(42);` — a composite value, later reassigned via `setValue(29)`

## Method Invocations

| Test | Method Call | Result |
|---|---|---|
| Instance A — getValue() | `getValue()` | 17 |
| Instance A — isEven(10) | `isEven(10)` | TRUE |
| Instance A — isOdd(10) | `isOdd(10)` | FALSE |
| Instance A — isEven(7) | `isEven(7)` | FALSE |
| Instance A — isOdd(7) | `isOdd(7)` | TRUE |
| Instance A — isPrime() on stored value 17 | `isPrime()` | TRUE |
| Instance B — getValue() | `getValue()` | 42 |
| Instance B — isEven(0) | `isEven(0)` | TRUE |
| Instance B — isOdd(-3) | `isOdd(-3)` | TRUE |
| Instance B — isPrime() on stored value 42 | `isPrime()` | FALSE |
| Instance B — setValue(29), then getValue() | `setValue(29) → getValue()` | 29 |
| Instance B — isPrime() on new stored value 29 | `isPrime()` | TRUE |

## isPrime() Sweep: 1 – 20

Each candidate is wrapped in its own instance of `BrittaneyMyInteger` and probed with all three boolean methods. Prime rows are highlighted so the classic prime sequence (2, 3, 5, 7, 11, 13, 17, 19) is easy to verify at a glance.

| n | isEven(n) | isOdd(n) | isPrime() |
|---|---|---|---|
| 1 | FALSE | TRUE | FALSE |
| 2 | TRUE | FALSE | TRUE |
| 3 | FALSE | TRUE | TRUE |
| 4 | TRUE | FALSE | FALSE |
| 5 | FALSE | TRUE | TRUE |
| 6 | TRUE | FALSE | FALSE |
| 7 | FALSE | TRUE | TRUE |
| 8 | TRUE | FALSE | FALSE |
| 9 | FALSE | TRUE | FALSE |
| 10 | TRUE | FALSE | FALSE |
| 11 | FALSE | TRUE | TRUE |
| 12 | TRUE | FALSE | FALSE |
| 13 | FALSE | TRUE | TRUE |
| 14 | TRUE | FALSE | FALSE |
| 15 | FALSE | TRUE | FALSE |
| 16 | TRUE | FALSE | FALSE |
| 17 | FALSE | TRUE | TRUE |
| 18 | TRUE | FALSE | FALSE |
| 19 | FALSE | TRUE | TRUE |
| 20 | TRUE | FALSE | FALSE |